

Chapter 6: Directives

1. Overview

A directive is a class that attaches behavior to elements in the DOM. Components *are* directives — specifically, directives with a template. Angular formally splits directives into three categories:

- **Components** — directives with a view (`@Component` extends the directive metadata with `template` / `templateUrl`).
- **Attribute directives** — change the appearance or behavior of an existing DOM element, component, or another directive (`NgClass`, `NgStyle`, `RouterLink`, a custom `[appHighlight]`).
- **Structural directives** — change the DOM layout by adding, removing, and manipulating elements and their surrounding `<ng-template>` (`*ngIf`, `*ngFor`, `*ngSwitchCase`).

Directives are Angular's mechanism for reusing DOM-manipulation logic without duplicating template markup or writing imperative DOM code. They are dependency-injectable, participate in change detection, have their own lifecycle hooks, and — since Angular 15 — can be composed into other directives via the **Directive Composition API**.

This chapter covers: writing attribute and structural directives from scratch, how the `*` syntax desugars to `<ng-template>`, `TemplateRef` / `ViewContainerRef` mechanics, host bindings/listeners, `exportAs`, directive composition (`hostDirectives`), the built-in structural/attribute directives, and the standalone-directive model introduced in Angular 14+ / made default in v19.

2. Core Concepts

2.1 Directive vs Component

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {}

@Component({ selector: 'app-widget', template: `<p>widget</p>` })
export class WidgetComponent {}
```

Internally, `@Component` is `@Directive` plus a compiled template (`@comp` vs `@dir` definitions). A component can only be applied where its selector matches an *element* it creates; a directive attaches to an *existing* host element created by something else (a component's template or plain HTML).

2.2 Attribute Directives

An attribute directive does **not** add or remove elements — it modifies the element it's attached to: its properties, attributes, classes, styles, or behavior in response to events.

Key building blocks:

- `@Input()` to accept configuration.
- `@HostBinding()` / `host: {}` metadata to bind to host element properties.

- `@HostListener()` / `host: {}` metadata to listen to host events.
- `ElementRef` injected to get a raw reference to the host DOM node (use `Renderer2` to mutate it, not direct DOM API, to stay server/Web-Worker safe).

Writing a custom attribute directive from scratch

```
import { Directive, ElementRef, HostListener, Input, Renderer2, inject } from
 '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true, // default in v19+, harmless to be explicit
})
export class HighlightDirective {
  private el = inject(ElementRef);
  private renderer = inject(Renderer2);

  @Input('appHighlight') highlightColor = 'yellow';
  @Input() defaultColor = '';

  @HostListener('mouseenter')
  onMouseEnter() {
    this.setColor(this.highlightColor || this.defaultColor || 'yellow');
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.setColor(null);
  }

  private setColor(color: string | null) {
    if (color) {
      this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', color);
    } else {
      this.renderer.removeStyle(this.el.nativeElement, 'backgroundColor');
    }
  }
}
```

Notes:

- The selector `[appHighlight]` is an **attribute selector**, matching `<div appHighlight>`.
- Aliasing the `@Input()` to the same name as the selector (`appHighlight`) enables the common "selector doubles as input" pattern: `<div [appHighlight]="orange">`.
- `Renderer2` is preferred over `el.nativeElement.style...` because it is abstracted from the rendering target (DOM, server-side rendering, web workers) and sanitizes some operations.

2.3 Structural Directives

Structural directives change the DOM *structure* — they add/remove/repeat elements. They are recognizable by the leading `*` in templates: `*ngIf`, `*ngFor`, `*ngSwitchCase`. The `*` is syntactic sugar that Angular's template compiler desugars into an `<ng-template>` wrapping the host element (see §4.1).

A structural directive needs two collaborators injected via DI:

- `TemplateRef<T>` — a reference to the `<ng-template>` content (the "stamp") the directive controls.
- `ViewContainerRef` — a reference to the location in the DOM (a *container*, represented by a comment anchor node) where embedded views can be inserted/removed.

Writing a custom structural directive from scratch

A directive that behaves like `*ngIf` but only renders when a user has a given role (`*appHasRole="'admin'"`):

```
import {
  Directive, Input, TemplateRef, ViewContainerRef, inject,
} from '@angular/core';

@Directive({
  selector: '[appHasRole]',
  standalone: true,
})
export class HasRoleDirective {
  private templateRef = inject(TemplateRef<unknown>);
  private viewContainerRef = inject(ViewContainerRef);
  private currentUserRole = 'editor'; // normally injected from an AuthService
  private hasView = false;

  @Input() set appHasRole(role: string) {
    const shouldShow = role === this.currentUserRole;

    if (shouldShow && !this.hasView) {
      this.viewContainerRef.createEmbeddedView(this.templateRef);
      this.hasView = true;
    } else if (!shouldShow && this.hasView) {
      this.viewContainerRef.clear();
      this.hasView = false;
    }
  }
}
```

Usage:

```
<div *appHasRole="'admin'">Only admins see this.</div>
```

Requirements for a class to be usable as a structural directive:

1. Selector must be an attribute selector (`[appHasRole]`), matched against the synthesized `<ng-template>`.

2. There must be an `@Input()` whose name matches the directive selector (`appHasRole`) — this is what receives the expression after the colon-less `*dir="expr"` shorthand binds to.
3. `TemplateRef` and `ViewContainerRef` are injected to control template stamping.

Passing multiple inputs / template context (`*ngFor`-style "let" and "as")

Structural directives can expose a typed **context object** to the embedded view, enabling `let x = y` template variables:

```
export interface AppUnlessContext {
  $implicit: boolean;
  appUnless: boolean;
}

@Directive({ selector: '[appUnless]', standalone: true })
export class UnlessDirective {
  private templateRef = inject(TemplateRef<AppUnlessContext>);
  private vcr = inject(ViewContainerRef);
  private hasView = false;

  @Input() set appUnless(condition: boolean) {
    if (!condition && !this.hasView) {
      this.vcr.createEmbeddedView(this.templateRef, { $implicit: condition, appUnless:
condition });
      this.hasView = true;
    } else if (condition && this.hasView) {
      this.vcr.clear();
      this.hasView = false;
    }
  }

  // Enables compile-time type-checking of the template with `ngTemplateContextGuard`
  static ngTemplateContextGuard(dir: UnlessDirective, ctx: unknown): ctx is AppUnlessContext
  {
    return true;
  }
}
```

`$implicit` is the special context key bound when you write `let x` without `= propertyName`; other keys are bound with `let y = propertyName`.

```
<p *appUnless="isHidden as flag">Value was: {{ flag }}</p>
```

`*ngFor`'s context (`$implicit`, `index`, `count`, `first`, `last`, `even`, `odd`) is the canonical example of this pattern.

2.4 Host Bindings and `host` Metadata

Two equivalent styles exist for binding to the host element:

```
// Decorator style
@HostBinding('class.active') isActive = true;
@HostListener('click', ['$event'])
onClick(event: MouseEvent) { ... }
```

```
// Metadata style (preferred in modern Angular – statically analyzable, tree-shakes better)
@Directive({
  selector: '[appToggle]',
  host: {
    '[class.active]': 'isActive',
    '(click)': 'onClick($event)',
    '[attr.aria-pressed]': 'isActive',
  },
})
export class ToggleDirective {
  isActive = false;
  onClick(event: MouseEvent) { this.isActive = !this.isActive; }
}
```

The `host` object is preferred in the Angular style guide because it's declarative, colocated with other metadata, and doesn't rely on decorator reflection for every binding.

2.5 exportAs

By default a directive instance isn't accessible from the template via a local reference unless it declares `exportAs`, allowing `#ref="exportAsName"`:

```
@Directive({ selector: '[appTooltip]', exportAs: 'appTooltip' })
export class TooltipDirective {
  show() { /* ... */ }
}
```

```
<div appTooltip #t="appTooltip">
  <button (click)="t.show()">Show tooltip</button>
</div>
```

`NgForm`'s `exportAs: 'ngForm'` (`#f="ngForm"`) is the most common real-world usage.

2.6 Directive Composition API (Angular 15+)

The Directive Composition API lets a component or directive apply other directives to *itself* at the class level via `hostDirectives`, without the consumer needing to add them in the template. This solves the long-standing problem of wanting to "mix in" behavior (e.g., `CdkMenu`, `NgControl`-like validators) without inheritance or manual host-binding duplication.

```
@Directive({
  selector: '[appHighlight]',
  host: { '[class.highlighted]': 'true' },
})
```

```

export class HighlightDirective {
  @Input() color = 'yellow';
}

@Directive({ selector: '[appClickTrap]', standalone: true })
export class ClickTrapDirective {
  @Output() trapped = new EventEmitter<void>();
  @HostListener('click', ['$event'])
  onClick(e: Event) { e.stopPropagation(); this.trapped.emit(); }
}

@Component({
  selector: 'app-card',
  standalone: true,
  template: `<ng-content />`,
  hostDirectives: [
    { directive: HighlightDirective, inputs: ['color: highlightColor'], outputs: [] },
    ClickTrapDirective, // no aliasing needed → include as-is
  ],
})
export class CardComponent {}

```

```
<app-card highlightColor="orange" (trapped)="onTrap()">content</app-card>
```

Key rules:

- Host directives are instantiated **before** the host component/directive itself, and their inputs/outputs are private by default — the consuming component must explicitly re-expose them via `inputs: [...]` / `outputs: [...]` (optionally aliasing with `'internalName: publicName'`).
- Host directives cannot themselves have their own `hostDirectives` alias collisions silently ignored — if two host directives (or the host + a host directive) define the same input/output name without aliasing, this is a compile-time error.
- A `hostDirective` entry must be a directive class already decorated with `@Directive`; you cannot list a component.
- Content projection, DI (the host directive can inject the host component if needed via hierarchical injectors), and change detection all work exactly as if the host directive were applied manually in a template — it's a compiler-time convenience, not a virtual-DOM trick.

2.7 Built-in Attribute Directives

Directive	Purpose	Notes
<code>NgClass</code>	Add/remove CSS classes based on an object/array/string expression	<code>[ngClass]="{active: isActive, disabled: isDisabled}"</code>
<code>NgStyle</code>	Set inline styles from an object expression	<code>[ngStyle]="{color: c, 'font-size.px': size}"</code>

Directive	Purpose	Notes
<code>NgModel</code>	Two-way binding for template-driven forms	Requires <code>FormsModule</code>
<code>RouterLink</code> , <code>RouterLinkActive</code>	Navigation and active-link styling	From <code>@angular/router</code>

Modern Angular (v17+) recommends preferring plain class/style bindings over `NgClass` / `NgStyle` where possible because they're cheaper (no extra directive instance, differ algorithm) —

`[class.active]="isActive"` and `[class]="classExpression"`, `[style.color]="c"` — but `NgClass` / `NgStyle` remain necessary for dynamic multi-class/style objects computed at runtime.

2.8 Built-in Structural Directives

- `*ngIf` (with `else` template reference and the `as` alias for the resolved value) — being superseded by the built-in `@if` control-flow block (v17+), which is not a directive at all (it's compiled specially, no `TemplateRef` instantiation overhead per branch, better type narrowing).
- `*ngFor` (with `trackBy`, and micro-syntax context vars `index`, `first`, `last`, `even`, `odd`) — superseded by `@for` with mandatory `track`.
- `*ngSwitch` / `*ngSwitchCase` / `*ngSwitchDefault` — superseded by `@switch` / `@case` / `@default`.

The new `@if` / `@for` / `@switch` control-flow blocks are **not structural directives** — they're a distinct template syntax handled directly by the compiler/runtime (`@template` with specialized instructions), which is why they don't desugar to `<ng-template>` with an attribute the way `*ngIf` does, and why they don't require importing `CommonModule`. Interviewers sometimes conflate the two — it's worth being precise that structural directives and the new control flow are two different mechanisms that happen to solve overlapping problems.

3. Code Examples

3.1 `*ngIf` / `*ngFor` combined, with `trackBy` and `as`

```
<ul *ngIf="users$ | async as users; else loading">
  <li *ngFor="let user of users; trackBy: trackByUserId; index as i">
    {{ i }}: {{ user.name }}
  </li>
</ul>
<ng-template #loading>Loading...</ng-template>
```

```
trackByUserId(index: number, user: User): number {
  return user.id;
}
```

3.2 Custom two-way-bindable attribute directive (Banana-in-a-box)

```
@Directive({ selector: '[appCounter]', standalone: true })
export class CounterDirective {
  @Input() appCounter = 0;
  @Output() appCounterChange = new EventEmitter<number>();

  @HostListener('click')
  increment() {
    this.appCounter++;
    this.appCounterChange.emit(this.appCounter);
  }
}
```

```
<button [(appCounter)]="clicks">Clicks: {{ clicks }}</button>
```

The `[( )]` syntax requires an `@Input()` `X` paired with an `@Output()` named `Xchange`.

3.3 NgClass / NgStyle vs plain bindings

```
<!-- NgClass: good for a dynamic map of many conditional classes -->
<div [ngClass]="{ error: hasError, warning: hasWarning, 'is-loading': loading }"></div>

<!-- Plain binding: cheaper for a single condition -->
<div [class.error]="hasError"></div>

<div [ngStyle]="{ 'background-color': bg, 'font-weight': bold ? 'bold' : 'normal' }"></div>
<div [style.background-color]="bg"></div>
```

3.4 A directive consuming another via Directive Composition API and DI

```
@Directive({ selector: '[appFocusable]', standalone: true })
export class FocusableDirective {
  @HostBinding('tabindex') tabindex = 0;
}

@Directive({
  selector: '[appButtonBase]',
  standalone: true,
  hostDirectives: [FocusableDirective],
})
export class ButtonBaseDirective {
  private focusable = inject(FocusableDirective, { self: true });

  disable() { this.focusable.tabindex = -1; }
}
```

4. Internal Working

4.1 How `*ngIf` Desugars to `<ng-template>`

The template compiler treats any attribute prefixed with `*` as **microsyntax** that must be expanded before compilation. Given:

```
<div *ngIf="isVisible" class="card">{{ content }}</div>
```

The compiler rewrites this to:

```
<ng-template [ngIf]="isVisible">
  <div class="card">{{ content }}</div>
</ng-template>
```

General desugaring rule for `*directive="expression"`: the host element is moved inside a synthesized `<ng-template>`, and the expression is bound to the `@Input()` on the directive whose name equals the directive's attribute name (here, `ngIf` → `NgIf.ngIf`).

For `*ngFor="let user of users; trackBy: trackByUserId"`, the **microsyntax parser** additionally expands `let` bindings into template context variables and keyword pairs (`trackBy: fn`) into additional inputs (`ngForTrackBy`):

```
<ng-template ngFor let-user [ngForOf]="users" [ngForTrackBy]="trackByUserId">
  <li>{{ user.name }}</li>
</ng-template>
```

At the Ivy instruction level, `<ng-template>` compiles to a `ɵɵtemplate()` call that registers a template function (the "embedded view factory") at that location, but does **not** render anything by itself — it only creates an anchor. Rendering is entirely up to the structural directive's logic (calling `viewContainerRef.createEmbeddedView(templateRef)`), which is why `*ngIf="false"` results in a comment node (`<!--ng-container ...-->` / `<!--bindings=...-->`) in the DOM rather than a hidden element — the content was never instantiated, unlike `[hidden]` or `display:none` which just visually hide an existing node.

4.2 `TemplateRef` and `ViewContainerRef` Mechanics

- `TemplateRef<C>`: an immutable factory/blueprint for a chunk of view. Injecting `TemplateRef` in a directive placed on `<ng-template>` (or the synthesized one from `*`) gives you the compiled template ready to be "stamped" (instantiated) multiple times, each stamp being an independent `EmbeddedViewRef`.
- `ViewContainerRef`: represents the anchor point (a comment node, `<!--container-->`) in the render tree where 0..n views can be inserted as siblings. It offers `createEmbeddedView(templateRef, context?, index?)`, `createComponent()`, `insert()`, `move()`, `remove()`, `clear()`, `indexOf()`, `get()`, `length`.

When `*ngFor` runs, for each item in the iterable it calls

`viewContainerRef.createEmbeddedView(templateRef, context)`, producing a distinct `EmbeddedViewRef` per item, each with its own context object (`$implicit`, `index`, etc.) and its own change-detection subtree — but *all embedded views share the same compiled template function*, which is why `*ngFor` is efficient: the template is parsed/compiled once, only instantiated N times.

`NgIf`'s actual (simplified) source illustrates the pattern used throughout `@angular/common`:

```
@Directive({ selector: '[ngIf]' })
export class NgIf<T = unknown> {
  private context: NgIfContext<T> = new NgIfContext<T>();
  private thenTemplateRef: TemplateRef<NgIfContext<T>> | null = null;
  private elseTemplateRef: TemplateRef<NgIfContext<T>> | null = null;
  private thenViewRef: EmbeddedViewRef<NgIfContext<T>> | null = null;
  private elseViewRef: EmbeddedViewRef<NgIfContext<T>> | null = null;

  constructor(private _viewContainer: ViewContainerRef, templateRef:
TemplateRef<NgIfContext<T>>) {
    this.thenTemplateRef = templateRef;
  }

  @Input() set ngIf(condition: T) {
    this.context.$implicit = this.context.ngIf = condition;
    this._updateView();
  }

  private _updateView() {
    if (this.context.$implicit) {
      if (!this.thenViewRef) {
        this._viewContainer.clear();
        if (this.thenTemplateRef) {
          this.thenViewRef = this._viewContainer.createEmbeddedView(this.thenTemplateRef,
this.context);
        }
      }
    } else {
      if (!this.elseViewRef) {
        this._viewContainer.clear();
        if (this.elseTemplateRef) {
          this.elseViewRef = this._viewContainer.createEmbeddedView(this.elseTemplateRef,
this.context);
        }
      }
    }
  }
}
```

This is exactly the pattern demonstrated in §2.3.

4.3 Directive Instantiation and Injection Order

For a single host element, Angular can apply *multiple* directives (matched by selector) plus at most one component. During template compilation, Ivy generates a `directiveDefs` array (in selector-match order) attached to the element's `TVIEW` node. At view creation:

1. Angular walks the matched directive list and instantiates each directive class via the injector *in the order they were matched* (component, if present, is instantiated as part of the same array but conventionally listed first if its selector matches the element).
2. Each directive's constructor-injected dependencies are resolved through the **hierarchical injector** for that node — this is how `NgModel` can inject `NgControl`, or how a directive can inject a sibling directive on the same host via `@Optional()` `@Self()`.
3. `ngOnChanges` / `ngOnInit` fire for all directives on a node in the same matched order, only after all directive instances on that node exist (so cross-directive `@Input()` values set via template bindings are available in `ngOnInit`, and directives can inject each other in constructors, but not rely on the other's `ngOnInit` having run yet unless order is guaranteed).
4. Host bindings (`host: {}` / `@HostBinding`) are applied in a dedicated "host bindings" phase per directive, run in matched order — this is why binding the same host property in two co-applied directives without care can produce order-dependent, hard-to-debug results (see Edge Cases below).

Directive matching itself happens statically at compile time: the compiler reads each directive's `selector` and, for each element in a template, computes the set of directives whose selector matches that element's tag/attributes/classes, embedding the match directly into the compiled instruction set (`@directiveInject` calls) — there's no runtime selector-matching cost.

4.4 Directive Composition API Internals

`hostDirectives` is resolved entirely at **compile time**, not runtime reflection:

1. When the compiler processes a component/directive's `hostDirectives` array, it flattens the list of host directive classes (recursively, if a host directive itself has `hostDirectives`) into the enclosing element's `directiveDefs`, positioned *before* the host directive/component's own definition.
 2. Each host directive is instantiated with its own instance and its own DI context — it is a full directive instance, not a mixin/merge of properties. Angular does not use prototype composition; the "composition" is purely at the template-instruction level (multiple directive instances applied to the same node), which is the same mechanism `[ngClass]` `[ngStyle]` co-application on one element already always used.
 3. The `inputs` / `outputs` remapping arrays are compiled into an alias table so that when the *host's* template consumer writes `<app-card highlightColor="...">`, the compiled binding instruction targets the *host directive's* `color` input directly — there's no proxying property on the host component itself at runtime.
 4. Because host directives are separate instances, they get their own change-detection participation, their own `ngOnDestroy`, and can be retrieved via `injector.get(HostDirectiveClass, null, {self: true})` from within the host component, but are **not** visible to `@ViewChild` / `@ContentChild` queries on their type from *outside* the host unless the host explicitly exposes something — the composition is intentionally encapsulated (opaque to consumers beyond the exposed inputs/outputs).
-

5. Edge Cases & Gotchas

- **Only one structural directive per element.** `<div *ngIf="x" *ngFor="let i of items">` is a compile error ("Can't have multiple template bindings on one element"). Use `<ng-container>` to nest: wrap one in an `<ng-container *ngIf="x">` around the `*ngFor` element (or, in v17+, just nest `@if / @for` blocks — no such restriction exists there).
- ***ngIf destroys and recreates, it doesn't hide.** Component state inside an `*ngIf="false"` branch is lost (constructor/ `ngOnInit` re-runs when it flips back true) — unlike `[hidden] / CSS display:none`, which preserve component instance and state. This is a frequent interview trap: "why did my form data reset when I toggled visibility?"
- **NgClass / NgStyle are comparatively expensive.** Each invocation does an object diff (`keyValueDiffers`) every change-detection cycle; for a single boolean class/style, `[class.foo] / [style.prop]` bindings are cheaper and preferred by the style guide.
- **trackBy and object identity.** Omitting `trackBy` in `*ngFor` (or `track` in `@for`) over an array that gets a new reference each time (e.g., from a fresh HTTP response) causes Angular to destroy and recreate *all* DOM nodes/component instances even if the data is logically the same — killing animations, input focus, and perf. `@for` in v17+ makes `track` mandatory specifically to force developers to confront this.
- **Directive/component input timing race.** If a host directive's input depends on the enclosing component's constructor-time state, remember instantiation order: host directives are constructed before the host component, so a host directive cannot inject the host component's fully-initialized state in *its own* constructor — only after `ngOnInit`, since Angular instantiates first, then runs `ngOnChanges / ngOnInit` across all directives on the node.
- **Selector collision ambiguity.** Two directives with overlapping attribute selectors (`[appFoo]` and `[appFoo][bar]`) matching the same element both apply — directives are additive, not overriding, so both instances run, and if both bind the same host property without namespacing, last-applied-wins is compiler-order-dependent and fragile.
- **exportAs name collisions.** If two applied directives export the same `exportAs` name, `#ref="name"` resolves ambiguously (compile-time error in current Ivy versions) — always pick distinct `exportAs` values in a shared codebase.
- **ViewContainerRef injected on a component vs on <ng-template>.** Injecting `ViewContainerRef` in a component gives you the container *at that component's own location* in its parent (useful for dynamically adding siblings next to yourself), which is a different concept from a structural directive's `viewContainerRef`, which represents the anchor left behind by the `*` syntax. Confusing "container of me" with "container I control for embedded views" is a common source of bugs when building dynamic-component-loading features.
- **Directive Composition API cannot compose components.** `hostDirectives` entries must be plain directives (`@Directive`), not `@Component` — you cannot mix in a full component's template this way; use content projection or dynamic component creation instead.
- **hostDirectives inputs are private unless re-exposed,** and if you forget to list an input/output in the `inputs / outputs` array, template bindings on the host element silently fail to reach the host directive (no compiler error — it's just not a recognized binding target on the host's own selector), leading to "my binding does nothing" bugs.
- **Standalone directives must be imported explicitly** wherever used (component's `imports: []` array)

— a very common migration/runtime error ("NG0304: 'appHighlight' is not a known attribute") from forgetting to add the directive to `imports` after converting to standalone.

6. Interview Questions & Answers

Q1. What's the difference between a component and a directive?

A component is a directive with an attached view (template + styles); it can only match elements it creates or elements it's the sole "owner" of via its own selector as a tag. A directive (attribute or structural) attaches to an already-existing host element/component and modifies its appearance, behavior, or the surrounding DOM structure without owning a template of its own (except structural directives, which *control* a template via `TemplateRef`, but don't define it themselves).

Q2. What are the three categories of directives in Angular, and how do you tell them apart in a template?

Components (custom element tag, e.g. `<app-foo>`), attribute directives (attribute-style binding on an existing element, e.g. `[appHighlight]`), and structural directives (recognized by the leading `*`, e.g. `*ngIf`, or written as an explicit `<ng-template>`).

Q3. Why does `*ngFor` need a `TemplateRef` and a `ViewContainerRef`?

`TemplateRef` is the compiled, reusable blueprint of the repeated element's markup; `ViewContainerRef` is the anchor location where instantiated copies ("embedded views") get inserted. `*ngFor` calls `viewContainerRef.createEmbeddedView(templateRef, context)` once per array item, producing N independent view instances from one compiled template, each with its own context object (`$implicit`, `index`, `first`, `last`, etc.).

Interviewer intent: checks that the candidate understands structural directives don't "loop over HTML" magically — they're explicit imperative view-factory calls, and that the candidate can connect this to why `*ngFor` performance depends on `trackBy` (reusing embedded views vs. destroy/recreate).

Q4. Explain exactly how `*ngIf="condition"` desugars.

The template compiler rewrites `<div *ngIf="condition">...</div>` into `<ng-template [ngIf]="condition"><div>...</div></ng-template>`. The `<div>` and its subtree become the template content owned by a synthesized `<ng-template>`; the expression after `=` is bound to the `NgIf` directive's `ngIf @Input()`. `NgIf` itself decides, based on the input's truthiness, whether to call `createEmbeddedView` or `clear()` on its injected `ViewContainerRef`.

Q5. Why does content inside `*ngIf="false"` completely disappear from the DOM (not just become hidden), and what's the practical implication?

Because `*ngIf` doesn't toggle visibility — it destroys the embedded view entirely (`viewContainerRef.clear()`), running `ngOnDestroy` on any components/directives inside, and removing their DOM nodes, leaving only a comment anchor. Practical implication: component/form state inside an `*ngIf` block is lost when it becomes false and reinitialized from scratch when it becomes true again — unlike `[hidden]` or `[style.display]`, which just toggle CSS and preserve the underlying component instance and state.

Q6. How would you write a custom structural directive equivalent to `*ngIf` but inverted (`*appUnless`)?

Inject `TemplateRef` and `ViewContainerRef` in the constructor. Expose an `@Input() appUnless` setter (name must match the selector). Inside the setter, when the condition is falsy and no view exists yet, call

`viewContainerRef.createEmbeddedView(templateRef)` and track a `hasView` flag; when truthy and a view exists, call `viewContainerRef.clear()` and reset the flag. (Full code in §2.3.)

Q7. What rule governs which `@Input()` receives the value in `*directive="expression"` shorthand?

The microsyntax binds the expression to the `@Input()` whose property name exactly matches the directive's selector name. For `*appHasRole="'admin'"` with selector `[appHasRole]`, the value `'admin'` binds to `@Input() appHasRole`. Additional `key: value` pairs after semicolons (`*ngFor="let x of items; trackBy: fn"`) bind to inputs named `<selector><key>` in camelCase (`ngForTrackBy`).

Q8. What is the Directive Composition API, and what problem does it solve?

Introduced in Angular 15 via the `hostDirectives` metadata property, it lets a directive/component apply other directives to itself declaratively, so consumers automatically get that behavior without adding it in their template. It solves the "I want to compose behavior without inheritance or without forcing every template consumer to remember to add three directives every time" problem — e.g., Angular CDK's `CdkMenuTrigger`-style composition, or a design-system button that always needs focus-management + ripple + a11y attributes bundled in.

Interviewer intent: distinguishes candidates who've only used directives from those who've built a component library or worked with Angular CDK/Material internals, where this API is heavily used (`CdkMenu`, `CdkListbox`, etc., in Angular 15+).

Q9. Are a host directive's inputs and outputs automatically available on the host component's selector?

No — by default they're private to the composition. The host must explicitly re-expose them via the `inputs` / `outputs` arrays in the `hostDirectives` entry (optionally with `'internalName: publicAlias'` renaming). If not listed, template bindings targeting that name on the host element's tag are not recognized bindings and silently fail to connect.

Q10. When would you prefer `[class.active]="cond"` over `[ngClass]="{active: cond}"`?

For a single conditional class, the plain `[class.x]` binding is cheaper — it's a direct binding Angular applies without going through `NgClass`'s internal `keyValueDiffer`-based object diffing, which runs every change-detection cycle to detect added/removed/changed keys even if nothing changed. `NgClass` earns its cost when you need to toggle many classes at once from a single dynamically-computed object/array, or the class names themselves are dynamic strings.

Q11. Can one element have multiple structural directives, e.g. `*ngIf` and `*ngFor` together?

No — Angular disallows more than one structural directive (more precisely, more than one `*`-syntax template binding) per element, because each desugars to wrapping the element in its own `<ng-template>`, and you can't wrap the same element in two templates simultaneously. The fix is nesting via `<ng-container>`: put `*ngIf` on an `<ng-container>` wrapping an inner element that carries `*ngFor`. The new `@if` / `@for` block syntax (v17+) doesn't have this restriction since blocks nest naturally without needing a host element per condition.

Q12. How does Angular decide the order in which multiple directives applied to the same host element are instantiated, and why does that matter?

Order is determined at compile time by the order directives are matched against the element's selector (typically declaration/import order feeding into the generated `directiveDefs` array for that `TView` node), and instantiation, `ngOnChanges`, and `ngOnInit` all run across co-located directives in that matched order. It matters when directives depend on each other via DI in the constructor (a later-in-order directive can inject an earlier one via `@self()`, but not vice versa reliably) or when multiple directives bind the same host

property/attribute — the host-bindings phase applies in matched order too, so ordering can silently determine which value "wins" if bindings collide.

Interviewer intent: probes whether the candidate has debugged a real multi-directive collision (e.g., two directives both setting `[attr.tabindex]`) rather than just knowing directives exist in isolation.

Q13. What is `exportAs` for, and give a built-in example.

`exportAs` lets a directive be captured by a template reference variable (`#ref="exportAsName"`) so other parts of the same template can call its public methods/read its properties, distinct from local refs on plain elements (`#e1`) which just give you the native `HTMLElement`. Built-in example: `NgForm` declares `exportAs: 'ngForm'`, enabling `<form #f="ngForm">` so `f.valid`, `f.value`, etc. are accessible in the template.

Q14. Why is `Renderer2` recommended over direct `ElementRef.nativeElement` manipulation inside a directive?

`Renderer2` abstracts DOM mutation behind a platform-agnostic API, so the same directive works correctly under server-side rendering (Angular Universal) and web-worker rendering where there is no real `document` / DOM to mutate directly; it also centralizes some sanitization/security behavior. Direct `nativeElement` manipulation works only in a real browser DOM context and bypasses Angular's abstraction layer, risking runtime errors or silently-wrong behavior outside a standard browser environment.

Q15. What is the practical difference between the legacy `*ngIf` / `*ngFor` / `*ngSwitch` directives and the newer `@if` / `@for` / `@switch` control-flow blocks (Angular v17+)?

The new blocks are not directives at all — they're first-class template syntax compiled to dedicated Ivy instructions, so they don't require importing `CommonModule` / `NgIf` / `NgFor`, don't allocate a directive instance or go through `TemplateRef` / `ViewContainerRef` public APIs the way user code would, and support things structural directives can't easily express, like `@for ... { } @empty { }` and mandatory `track` expressions (Angular's answer to the frequently-forgotten `trackBy`). They also allow multiple "structural-like" blocks nested without the "one structural directive per element" restriction, since blocks aren't tied to a single host element the way `*` syntax is.

7. Quick Revision Cheat Sheet

- **3 kinds of directives:** components (view), attribute (modify existing element), structural (add/remove/repeat DOM via `<ng-template>`).
- **Attribute directive essentials:** `@Directive({selector: '[x]'})`, `@Input()`, `host: {}` (or `@HostBinding` / `@HostListener`), inject `ElementRef` + `Renderer2` for safe DOM mutation.
- **Structural directive essentials:** attribute selector, an `@Input()` named exactly like the selector, inject `TemplateRef` + `ViewContainerRef`, call `createEmbeddedView()` / `clear()`.
- **`*directive="expr"` desugars to** `<ng-template [directive]="expr">...</ng-template>`; `let x` → context `$implicit` / named keys; `key: val` → `directivekey` input.
- **Only one `*` structural directive per element** — nest with `<ng-container>`, or switch to `@if` / `@for` / `@switch` (no such limit).
- **`*ngIf` destroys/recreates** (state lost); `[hidden]` / `[style.display]` only toggle visibility (state preserved).
- **`exportAs` enables** `#ref="name"` template variables bound to the directive instance, not the native element.

- **Directive Composition API** (`hostDirectives`, **v15+**): compile-time mixing of directive behavior into a host; inputs/outputs private by default, must be re-exposed via `inputs` / `outputs` arrays; cannot compose full components, only `@Directive` classes; host directives instantiate before the host itself.
- **Multiple co-located directives** instantiate/init/apply host bindings in compiler-matched order — order matters for cross-directive DI and for binding collisions on the same host property.
- `NgClass` / `NgStyle` = dynamic multi-key diffing (costlier); prefer `[class.x]` / `[style.y]` for single static-name conditions.
- `@if` / `@for` / `@switch` (**v17+**) are compiled template syntax, not directives — no `CommonModule` import needed, no per-branch `TemplateRef` instantiation overhead, `track` mandatory in `@for`.
- **Standalone directives** must be added to the consuming component's `imports: []` array explicitly — missing this causes "not a known attribute" template errors.

Created By - Durgesh Singh